전자회로실험 · STM32 실습

STM32_2
컴퓨터와 FPGA 통신

예비보고서

실험일 2026 년 10 월 26 일

작성자 주윤찬

학번 2023440132

작성일 2026 년 10 월 26 일

과제 5~7 / FPGA 서보모터·SRAM·DAC

GitHub: YunchanJu/lab4_stm32_0

제출 태그: stm32-2-prelab-20261026-v1

1. 실험 목표와 MCU 구성

1.1 단계별 목표

실습은 PC 의 문자 입력을 STM32 에서 수신한 뒤 FPGA 까지 전달하고, 명령에 따라 FPGA LED 를 제어하는 과정이다. 과제 5→6→7 순서로 한 단계의 입력·출력을 확인한 후 다음 단계를 수행한다. FPGA 개별 실습은 서보모터, SRAM, DAC 순서이다.[1]

과제	데이터 흐름	기대 결과
5	PC→MCU→PC	문자와 16 진수 일치
6	PC→MCU→FPGA→PC	수신 바이트 동일
7	PC→MCU→FPGA	0/1/2 명령 LED 제어

1.2 통신 조건과 예상 시간

Serial Monitor 와 MCU UART2·UART3, FPGA UART 를 9600 baud, 8 비트, 패리티 없음, 정지 비트 1(8N1)로 맞춘다. 한 문자를 보낼 때 줄바꿈은 None 이다. 입력이 없으면 getchar()가 대기하는 것이 정상이다.[1]

한 비트는 1/9600=104.167μs 이며 시작 1+데이터 8+정지 1 의 10 비트 전송은 약 1.04167ms 이다.
PC→MCU→FPGA 에는 수신·전달·송신 처리 시간이 더해지므로 이를 단일 비트 시간과 혼동하지 않는다.

입력 문자	바이트 값
'1'	0x31
'A'	0x41
'a'	0x61
숫자 1	0x01: 문자와 다름

NUCLEO-G474RE 의 ST-LINK/VCP 연결은 보드 사용자 매뉴얼 UM2505 와 교안에 따라 확인한다.[1,3]

1.3 작성한 C 코드

```
int ch = getchar();
if (ch == EOF) return;
uint8_t data = (uint8_t)ch;
int printable=(data>=32U&&data<=126U);
printf("RX char=%c hex=0x%02X\r\n",
    printable ? (char)data : '.',
    (unsigned)data);
#if STM32_2_STAGE >= 6
HAL_StatusTypeDef result =
  HAL_UART_Transmit(&huart3,
                    &data,1U,100U);
#endif
```

EOF 를 표현하기 위해 getchar 결과를 int 로 받고, 정상 입력만 uint8_t 로 변환한다. 출력 시 비인쇄 문자는 점으로 표시하되 16 진수는 원래 바이트를 유지한다. UART2 인터럽트 수신과 getchar 를 동시에 사용하지 않는다.

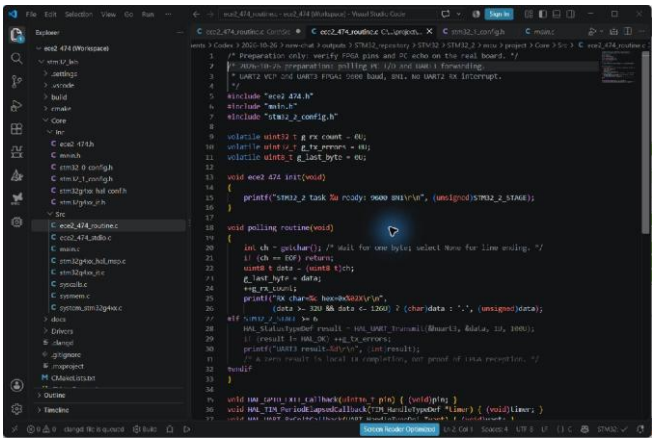


그림 1. UART 수신 및 FPGA 송신을 위한 C 코드.

STM32_2_STAGE 값으로 과제 5·6·7 의 동작을 선택한다. 세 모드의 ARM 빌드를 확인하였으며, g_rx_count·g_last_byte·g_tx_errors 로 수신 횟수, 마지막 바이트, 송신 오류를 관찰한다. 실험에서는 COM 포트로 전송한 문자와 출력 바이트를 비교할 예정이다.

전체 MCU 프로젝트·빌드 안내

2. 과제 6·7 · UART 전달과 FPGA 명령 처리

2.1 배선과 FPGA 핀 설정

MCU PC10(TX)을 FPGA 확장 GPIO RX 에, 양쪽 GND 를 공통으로 연결한다. FPGA TX 는 기존 F6 의 USB-UART 출력 경로를 유지하여 PC 에서 FPGA 수신값을 확인한다. MCU 와 FPGA 의 COM 포트 두 개를 각각 열어 비교한다.[1]

신호	연결·설정
MCU→FPGA	PC10 TX → 확장 GPIO RX
FPGA→PC	F6 TX → 보드 USB-UART
공통 접지	MCU GND ↔ FPGA GND
MCU 응답(선택)	별도 FPGA TX → PC11 RX
FPGA XDC	확장 GPIO 의 실제 PACKAGE_PIN 확인

FPGA 의 기존 RX C6 는 보드 내부 USB-UART 경로이다. MCU 의 PC10 을 연결할 확장 GPIO 를 핀맵에서 선택한 뒤, 해당 FPGA 패키지 핀을 XDC 의 RX 에 지정한다. MCU 의 PC10 은 IOC 에서 생성된 UART3 송신 설정을 사용한다.[1]

2.2 UART RTL 구성

새 수신 회로는 비동기 입력을 2 단 플립플롭으로 동기화하고 시작 비트 중심에서 재확인한다. 이후 8 비트를 LSB 부터 표본화하고 정지 비트가 HIGH 인지 확인하여 data_valid 를 한 클록 발생시킨다. 수신 바이트는 UART 송신기로 에코하고 LED 명령 처리에 함께 전달한다.

1 바이트 대기 버퍼와 overrun·framing_error 상태를 두었다. 이번 구현은 실습용 단일 스트림이다. 지속적인 고속 입력이나 장시간 PC 출력 정체는 별도 FIFO·흐름 제어가 필요한 조건이다.

UART RX/TX·LED RTL 과 TB

2.3 명령 및 자체 검사

문자	LED 동작	반복 입력 기대
0	끄기	계속 꺼짐
1	켜기	계속 켜짐
2	반전	매번 바뀜
기타	유지	현재 상태 유지

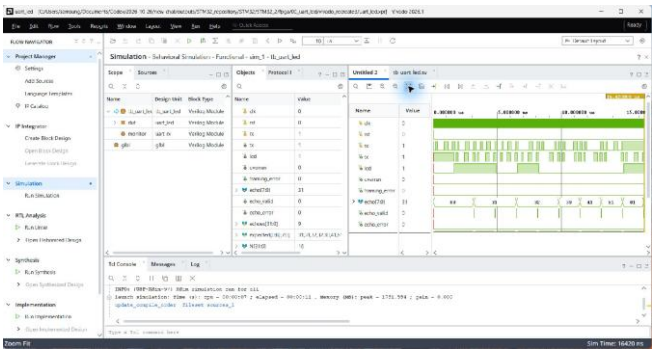


그림 2. UART 입력·에코·LED 제어 시뮬레이션.

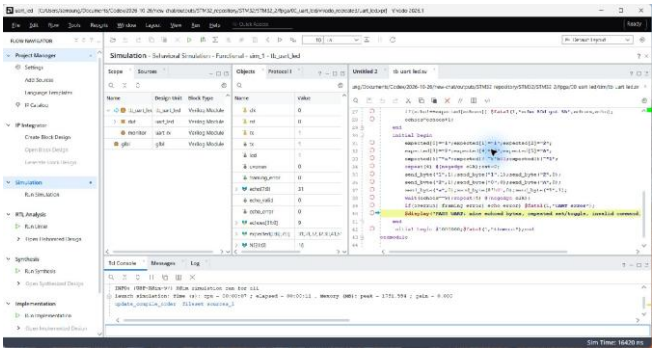


그림 3. 9 바이트 에코, 반복 명령 및 예외 입력 자체 검사 완료.

TB 는 문자 1 반복, 2 반복, 0, A, a, 숫자 0x01 및 최종 1 을 전송하여 에코와 LED 결과를 검사했다. Vivado 에서 9 개의 에코와 오류 신호 0 을 확인했다. 시뮬레이션은 시간을 줄인 분주 파라미터로 수행하며 파형 축의 µs 를 실제 9600 baud 의 실측 시간으로 해석하지 않는다.

MCU HAL 결과 0 은 MCU 송신 함수의 정상 종료를 뜻한다. FPGA 수신 성공은 FPGA USB-UART 출력과 실제 LED 로 따로 확인한다. 최종 시연은 1/A/a 바이트 비교와 0/1/2 반복 명령을 포함한다.

3. FPGA 서보모터 제어

3.1 구조와 설계값

서보 제어는 위치 갱신과 PWM 생성으로 구성한다.[2] 초기 펄스 폭은 1500μs 이며 좌·우 입력마다 100μs 씩 변경하고 1000~2000μs 범위로 제한한다. PWM 은 20ms 프레임을 유지하며 새 폭은 프레임 경계에서 적용한다.

항목	예상값(50MHz)
클록 주기	20ns
프레임	1,000,000 클록 =20ms
1000μs HIGH	50,000 클록, 5%
1500μs HIGH	75,000 클록, 7.5%
2000μs HIGH	100,000 클록, 10%

```
FRAME_CYCLES = CLK_HZ/50;
if (frame_end)
  width_cycles <=
    pulse_us*(CLK_HZ/1000)/1000;
assign servo = !rst_p &&
               count<width_cycles;
```

버튼 입력은 모터 구동 핀에 직접 연결하지 않고 동기화·디바운스 후 한 클록 펄스로 전달해야 한다. 현재 핵심 모듈의 left_pulse/right_pulse 포트는 정제된 입력을 전제로 한다. 외부 모터 전원과 접지, 신호 레벨은 사용 장비 교재의 연결도를 확인한다.

3.2 실험 절차

초기 1500μs 를 계측한 다음 좌측 반복 입력으로 1000μs, 우측 반복으로 2000μs 까지 확인한다. 양쪽 동시 입력은 위치를 유지하고 리셋은 초기값으로 돌아와야 한다. 파형을 먼저 확인한 후 모터의 방향·이동을 영상으로 기록한다. 펄스 폭과 실제 각도의 대응은 사용 모터에서 측정한다.

3.3 사전 시뮬레이션

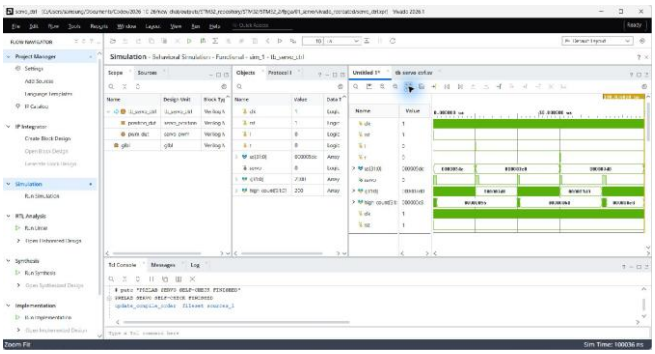


그림 4. 서보 펄스 폭 1500→1000→2000μs 의 설정 변화.

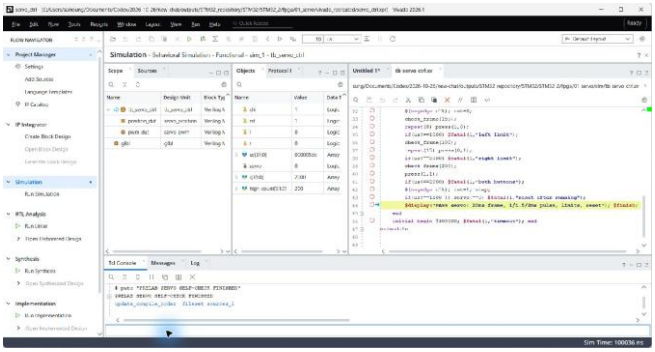


그림 5. 서보 프레임·폭·상하한·동시 입력·리셋 검사 완료.

TB 의 CLK_HZ 는 100000 으로 축소하고 프레임 2000 클록에서 HIGH 횟수 150·100·200 을 각각 검사했다. 이는 20ms 프레임과 1.5·1·2ms 펄스의 비율에 대응한다. 테스트 클록 자체도 빠르게 설정되므로 화면 시간축은 실기 계측값이 아니다.

펄스 폭 변경 후 새 프레임의 시작부터 한 주기를 세어 HIGH 길이를 비교하였다. 시뮬레이션에서 프레임 길이, 상·하한, 동시 입력과 리셋을 확인하였다. 실제 모터의 각도와 허용 펄스 범위는 보드 실험에서 점검할 예정이다.

서보 소스·TB·실행 Tcl·검증 로그

4. FPGA SRAM 읽기·쓰기

4.1 양방향 버스의 제어

SRAM 실습은 주소에 데이터를 기록한 뒤 같은 주소를 읽어 일치 여부를 확인한다. 교안의 SRAM 제어 모듈과 디지털 메모리 모델 TB 를 구성했다.[1,2] 쓰기 때는 FPGA 가 data bus 를 구동하고 읽기 때는 FPGA 가 버스를 Z 로 해제하여 외부 메모리가 구동하도록 한다.

상태	버스 구동	제어의 의미
쓰기	FPGA 출력	CS 활성, WE 활성, OE 비활성
읽기	FPGA Z	CS·OE 활성, WE 비활성
유휴	Z	장치 비선택
동시 요청	Z	충돌 방지, 비활성

n 접미사의 CS_n·OE_n·WE_n 은 LOW 활성이다. 출력 허용과 쓰기 요청을 동시에 활성화하면 양쪽이 버스를 구동할 수 있으므로 허용하지 않는다. 주소·데이터 setup/hold 시간은 실제 SRAM 데이터시트의 요구값에 맞춰야 한다.

4.2 검사 입력과 예상값

시뮬레이션에서는 주소 0~15 에 15-address 값을 기록한 뒤 같은 주소를 다시 읽어 비교한다. 주소 0 은 15, 주소 1 은 14, 주소 15 는 0 이 기대된다. 서로 다른 값을 사용하면 주소선과 데이터선의 오류를 찾기 쉽다.

주소	쓰기값	읽기 기대
0	0xF	0xF
1	0xE	0xE
8	0x7	0x7
15	0x0	0x0

실기에서는 주소/데이터 비트 폭과 커넥터 핀을 교재로 확인한다. 여러 주소를 기록한 후 읽고 쓰기 신호와 버스의 방향 전환을 파형·영상으로 함께 남긴다. 결과가 다르면 주소 오류와 데이터 오류를 분리하여 점검한다.

4.3 사전 검증 결과

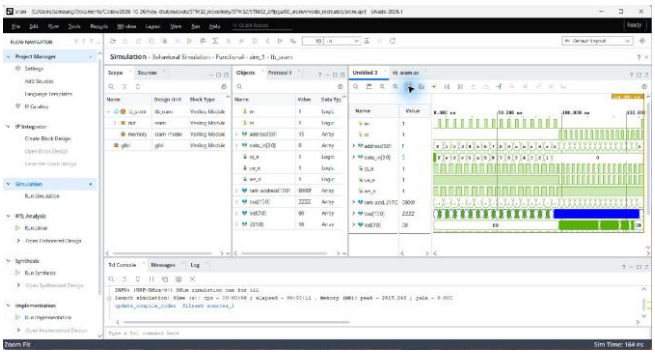


그림 6. 주소 진행, 쓰기·읽기 구간, 버스 방향의 Vivado 파형.

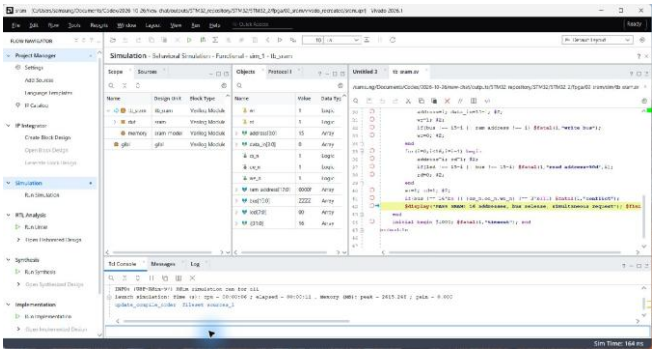


그림 7. 16 주소 일치·버스 해제·동시 요청 검사 완료.

실제 Vivado XSim 에서 16 개 주소의 쓰기·읽기 비교, 유휴 시 Z 상태, 동시 요청 시 제어 비활성을 모두 통과했다. 이때 사용한 메모리 모델은 기능 검사 모델이며 실제 외부 장치의 접근 지연·출력 해제 지연·전압 특성을 모두 재현하지 않는다.

실험에서 읽기값이 고정되면 OE/WE 의 활성 극성과 버스의 Z 전환을 먼저 점검한다. 특정 주소에서만 불일치하면 주소선과 장치 선택을 확인한다. 핀맵을 기준으로 XDC 를 확정하고, 실제 SRAM 의 접근 시간과 데이터 유지 조건을 검토한 후 동작을 측정한다.

SRAM 소스·TB·Vivado 결과

5. FPGA DAC 제어

5.1 디지털 코드와 아날로그 출력

DAC 실습은 FPGA 가 외부 DAC 에 코드를 전달하여 아날로그 출력을 만드는 실습이며 STM32 내부 DAC 를 사용하는 것이 아니다.[1] 교안의 병렬 DAC 제어 모듈을 구성하여 8 비트 램프 데이터와 CS_n·LDAC_n·WE_n, 채널 선택 A/B 를 출력한다.[2]

DAC 의 출력은 디지털 코드 N, 기준 전압 및 출력 회로에 따라 결정된다. 단극성 8 비트 DAC 의 이상적인 관계는 장치에 따라 N/256 에 비례하는 형태로 나타난다. 사용할 DAC 의 데이터시트로 기준 전압과 전달 특성을 확인한 뒤 코드별 예상 전압을 계산할 예정이다.

코드	확인 목적
0x00	영점 부근
0x40	하위 중간 구간
0x80	중간 코드
0xC0	상위 중간 구간
0xFF→0x00	최대 코드 및 랩어라운드

5.2 타이밍과 검사 항목

데이터와 채널은 쓰기 펄스 동안 안정되어야 한다. TB 는 UPDATE_CYCLES=4 로 빠르게 실행하여 260 번의 쓰기, 255→0 랩어라운드, 중간 채널 변경, 데이터 유지 및 리셋을 확인한다. 실제 장치에 연결할 때는 제어 신호 극성과 최소 펄스 폭·setup/hold 를 데이터시트에 맞춰 검토한다.

실기에서는 코드별 출력 전압을 측정하여 예상 전압과의 차이, 단조 증가, 램프의 주기 및 되돌아가는 구간을 기록한다. 채널 전환 전후에 출력 단자를 혼동하지 않고 GND 기준으로 계측한다.

5.3 DAC 시뮬레이션 결과

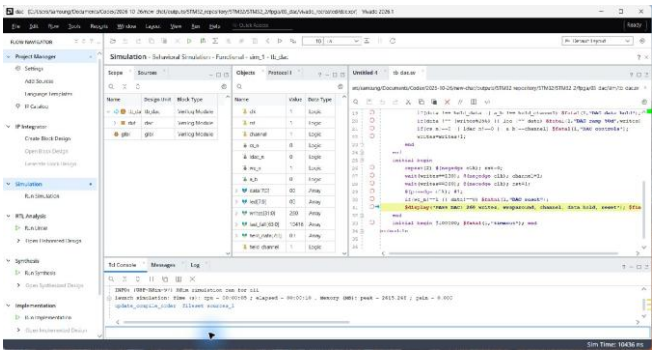


그림 8. DAC 260 쓰기·랩어라운드·채널·유지·리셋 PASS.

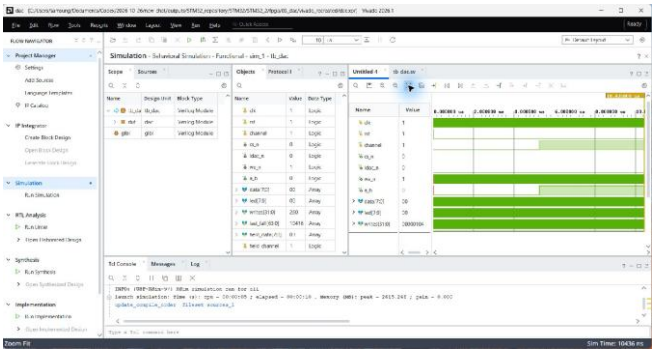


그림 9. 채널 전환을 포함한 전체 디지털 파형.

실제 Vivado GUI XSim 에서 자체 검사를 통과했다. TB 는 쓰기 펄스 구간에서 데이터와 채널이 변하지 않는지 확인한다. 디지털 코드가 증가하는 결과는 확인했지만 아날로그 전압·출력 settling·부하 효과는 모델에 포함하지 않았다.

출력 전압이 예상과 다르면 Vref, 출력 범위 설정, 채널 선택과 부하를 점검한다. 코드가 간헐적으로 잘못되면 쓰기 에지와 데이터 유지 시간을 확인한다. 실험에서는 각 코드의 실제 전압을 측정해 예상값과 비교할 예정이다.

DAC 소스·TB·검증 로그

6. 실행 계획·예상 결과 및 참고문헌

6.1 준비 상태 및 현장 확인

항목	사전 확인	실험 확인 항목
MCU5~7	ARM 빌드 3/3 PASS	COM 문자·바이트 화면
UART FPGA	9 바이트 XSim PASS	핀 확정·에코·LED 영상
서보	폭·제한 XSim PASS	펄스·위치 변화 시연
SRAM	16 주소 XSim PASS	외부 장치 읽기·쓰기
DAC	260 쓰기 XSim PASS	전압-코드 표·파형

실험 순서는 과제 5 단독 VCP 확인 → 과제 6 PC10 전달 및 FPGA COM 비교 → 과제 7 반복 명령 LED 제어 → 서보 → SRAM → DAC 이다. 각 단계의 소스 설정과 실제 배선이 일치하는지 확인하고 실행 화면을 남긴다.

6.2 오류 원인과 점검 방법

입력 대기는 getchar 의 정상 상태일 수 있다. 문자가 깨지면 양쪽 baud·8N1·COM 포트를 확인한다. MCU 결과 0 인데 FPGA 출력이 없으면 공통 GND, PC10 연결, FPGA RX XDC 가 C6 에 남아 있는지를 확인한다. 명령이 한 번에 여러 개 처리되면 Serial Monitor 줄바꿈 None 과 입력 바이트를 확인한다.

시뮬레이션 후에는 실제 배선과 신호 전압, 외부 장치의 시간 조건을 점검한다. UART 의 수신 바이트, 서보 펄스 폭, SRAM 읽기값, DAC 출력 전압을 차례로 측정하고 예상값과 비교할 예정이다.

6.3 예상 결론

예상 결과는 문자의 전 구간 일치, 명령별 LED 의 안정적 갱신, 서보 펄스 폭 제한, SRAM 주소별 일치, DAC 코드 증가에 따른 출력 변화이다. 실제 보드 결과와의 비교 및 오차 분석은 측정 후 결과보고서에 작성한다.

6.4 소스 코드 및 제출 버전

https://github.com/YunchanJu/lab4_stm32_0

제출 태그: **stm32-2-prelab-20261026-v1**
소스·검증 기준 커밋:
cf68ea5bc775d231aafca292da9399f31893b6d0

MCU 전체 프로젝트 · FPGA UART

서보 · SRAM · DAC

C 코드 및 FPGA 시뮬레이션 · 검증 요약

과제 5~7 의 C 코드와 UART/LED RTL, 서보·SRAM·DAC 의 설계 모듈 및 테스트벤치를 실험별 폴더에 구성하였다. 회로와 검사 방법은 참고문헌 [1,2]를 바탕으로 하였으며, 소스 코드와 시뮬레이션 결과는 아래 경로에서 확인할 수 있다.

참고문헌

[1] thechef030,「STM32_2 · 컴퓨터와 FPGA 통신」, 4 주 실습 매뉴얼. 2026-10-26 확인. 과제 5~7·연결 및 개별 실습 요구사항을 요약·참고하였다.
STM32_2 교안 원문

[2] thechef030,「FPGA 모듈별 Verilog 소스와 테스트벤치」, FPGA_additional.md. 서보·SRAM·DAC 기능별 모듈 및 TB 참고·사용. 2026-10-26 확인.
FPGA 소스 교안 원문

[3] STMicroelectronics, *STM32G4 Nucleo-64 boards (MB1367)*, User manual **UM2505**. 보드 및 ST-LINK/VCP 구성 참고.
ST 공식 사용자 매뉴얼

그림 1~9 는 UART 송수신 코드와 FPGA 의 UART·서보·SRAM·DAC 시뮬레이션 결과이다. 실제 보드 실험에서는 통신 바이트, 펄스 폭, 메모리 읽기값 및 출력 전압을 확인할 예정이다.